Validation Tests for $\texttt{avoid}_p oints_p ath$

1 Introduction

This document collects validation tests for the function $\texttt{avoid}_p oints_p ath$, $designedtoconstructdiscretepolygonalpati$

2 Auxiliary Metrics

2.1 Minimum Distance to Forbidden Set

```
function min_distance_to_set(p::Path, S)
    minimum(
        abs(v.z - a.z)
        for v in p.vertices
        for a in S
    )
end
```

2.2 Maximum Turning Angle

```
function max_turn_angle(p::Path)
     max   = 0.0

    for i in 2:length(p.vertices)-1
        v1 = p.vertices[i].z - p.vertices[i-1].z
        v2 = p.vertices[i+1].z - p.vertices[i].z

           = acos(
            real(conj(v1)*v2) /
            (abs(v1)*abs(v2))
        )

         max   = max( max ,    )
    end

     max
end
```

3 Tests

3.1 Basic Sanity Test

```
@testset "avoid_points_path basic" begin

    z0 = Vertex(0.0,0.0)
    zf = Vertex(5.0,0.0)

    S = [
        Vertex(2.5,0.5),
        Vertex(2.5,-0.5)
    ]

    p = avoid_points_path(z0,zf,S)

    @test startpoint(p) == z0.z
    @test endpoint(p)   == zf.z
    @test length(p) > 2
end
```

3.2 Safety Distance Test

```
@testset "safety distance" begin

    z0 = Vertex(0.0,0.0)
    zf = Vertex(6.0,0.0)

    S = [Vertex(3.0,0.0)]

    p = avoid_points_path(z0,zf,S)

    dmin = min_distance_to_set(p,S)

    @test dmin > 0.2
end
```

3.3 Obstacle Bypass

```
@testset "obstacle bypass" begin

    z0 = Vertex(0.0,0.0)
    zf = Vertex(4.0,0.0)

    S = [Vertex(2.0,0.0)]

    p = avoid_points_path(z0,zf,S)

    ys = [imag(v.z) for v in p.vertices]

    @test maximum(abs.(ys)) > 0.1
end
```

3.4 Smoothness Test

```
@testset "smoothness" begin

    p = avoid_points_path(
        Vertex(0,0),
        Vertex(5,0),
        [Vertex(2.5,0.0)]
    )

    @test max_turn_angle(p) < pi/2
end
```

3.5 Allowed Final Collision

```
@testset "final allowed collision" begin

    z0 = Vertex(0,0)
    zf = Vertex(3,0)

    S = [zf]

    p = avoid_points_path(z0,zf,S)

    hits =
        count(v -> v == zf, p.vertices)

    @test hits == 1
end
```

3.6 Random Stability Test

```
@testset "random stability" begin

for k in 1:20

    z0 = Vertex(0.0,0.0)
    zf = Vertex(5.0,0.0)

    S = [Vertex(rand()*4, rand()-0.5)
         for _ in 1:5]

    p = avoid_points_path(z0,zf,S)

    @test endpoint(p) == zf.z
end

end
```

4 Conclusion

These tests validate that the generated paths:

- reach the prescribed endpoint,
- avoid forbidden points,
- maintain safety distance,
- remain geometrically smooth,
- behave robustly under random configurations.

Such properties are essential for stable numerical lifting procedures in polynomial covering algorithms.